

虚拟内存-03

古金宇

上海交通大学并行与分布式系统研究所

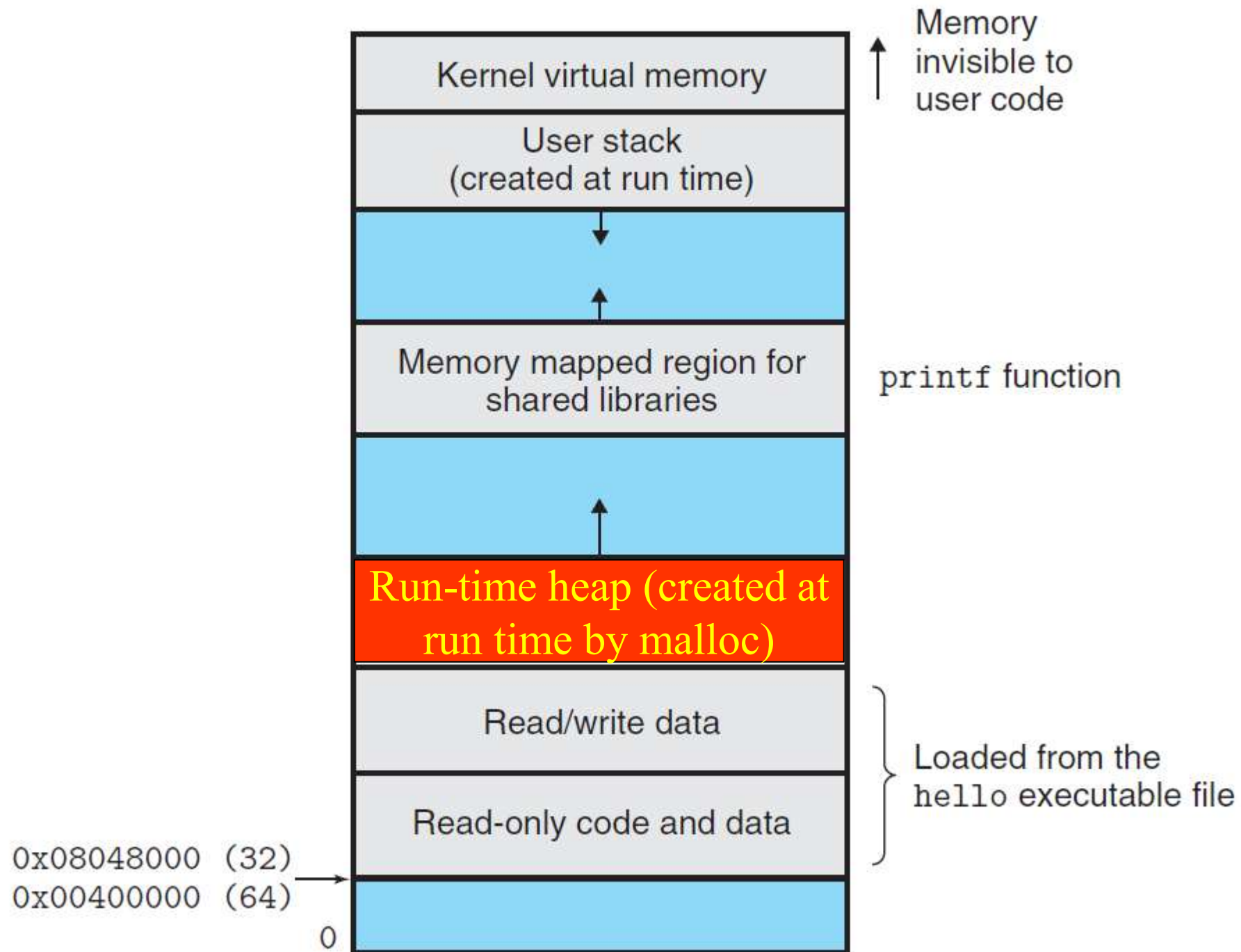
<https://ipads.se.sjtu.edu.cn>

Outline

- **The allocation functions**
- **Implementation of a simple allocator**
- **Explicit Free List**
- **Segregated Free List**
- **Buddy & Slab allocator**
- **Suggested reading: 9.9, 银杏书（物理内存章节）**

Dynamic Memory Allocation

- **Explicit vs. Implicit Memory Allocator**
 - Explicit: application allocates and frees space
 - E.g., `malloc` and `free` in C
 - Implicit: application allocates, but does not free space
 - E.g., garbage collection in Java, Python
 - E.g., compiler of Rust
- **Allocation**
 - In both cases, the memory allocators provide an abstraction of memory as a set of blocks
 - Dole out free memory blocks to application



Malloc-Related Interfaces

- `#include <stdlib.h>`
- `void *malloc(size_t size)`
 - if successful:
 - returns a pointer to a memory block of at least size bytes, aligned to 8-byte boundary (the memory is not cleared)
 - if `size == 0`, returns `NULL`
 - if unsuccessful: returns `NULL` (errno is set as `ENOMEM`)
- `void free(void *p)`
 - returns back the block pointed at by `p` to the pool of available memory
 - `p` must come from a previous call to **malloc**, **calloc** or **realloc** (otherwise, undefined behavior)

We leave a question here: Why undefined behavior?

Why Dynamic Memory Allocation

```
#define MAXN 15213
int array[MAXN];

int main(void)
{
    int i, n;
    scanf("%d", &n);
    if (n > MAXN)
        app_error("Input file too big");
    for (i = 0; i < n; i++)
        scanf("%d", &array[i]);
    exit(0);
}
```

The problem:

How to set a proper MAXN?

If not suitable, we need to modify the code and recompile the program.

Why Dynamic Memory Allocation

```
#include "csapp.h"

int main(void)
{
    int *array, i, n;

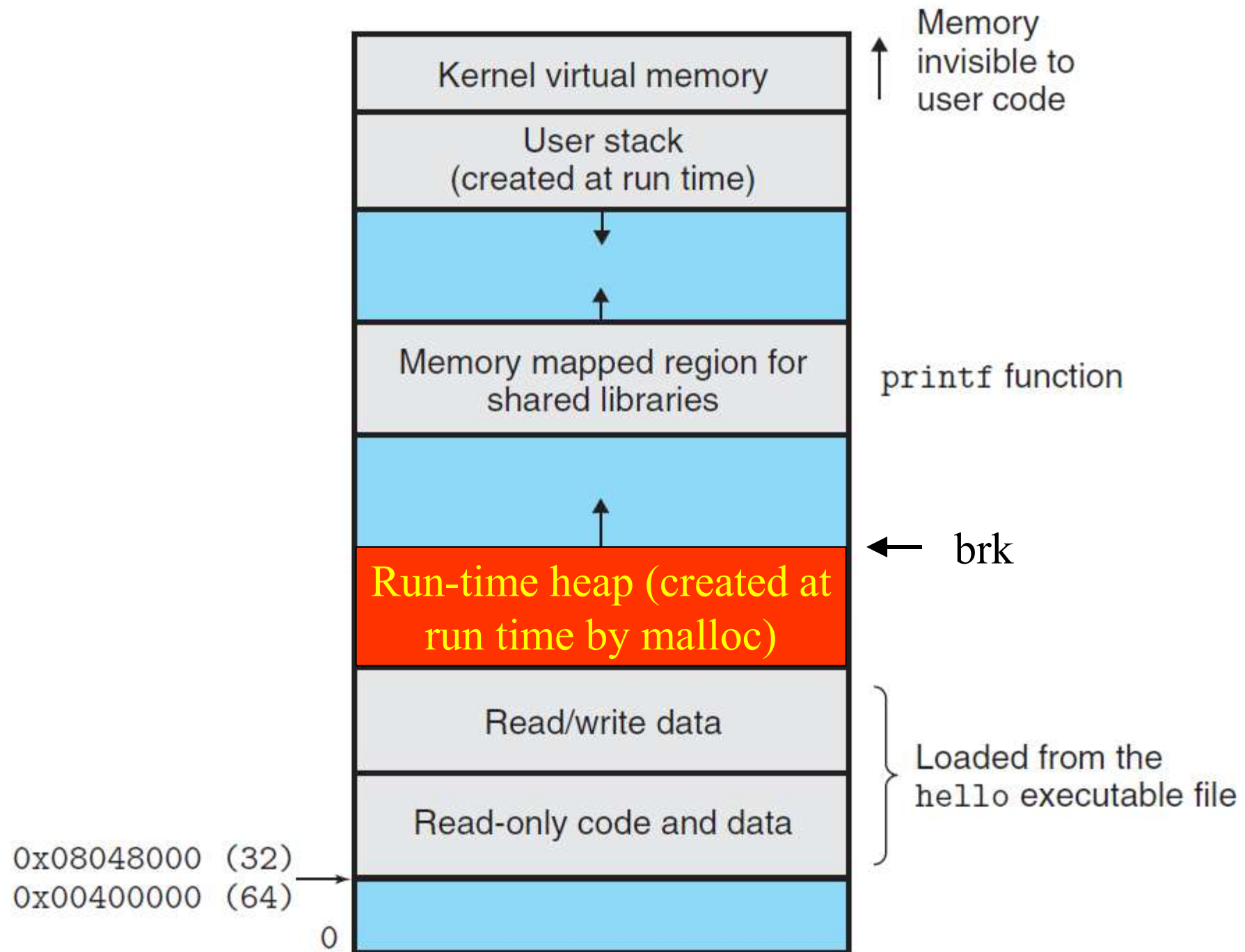
    scanf("%d", &n);
    array = (int *)Malloc(n * sizeof(int));
    for (i = 0; i < n; i++)
        scanf("%d", &array[i]);
    exit(0);
}
```

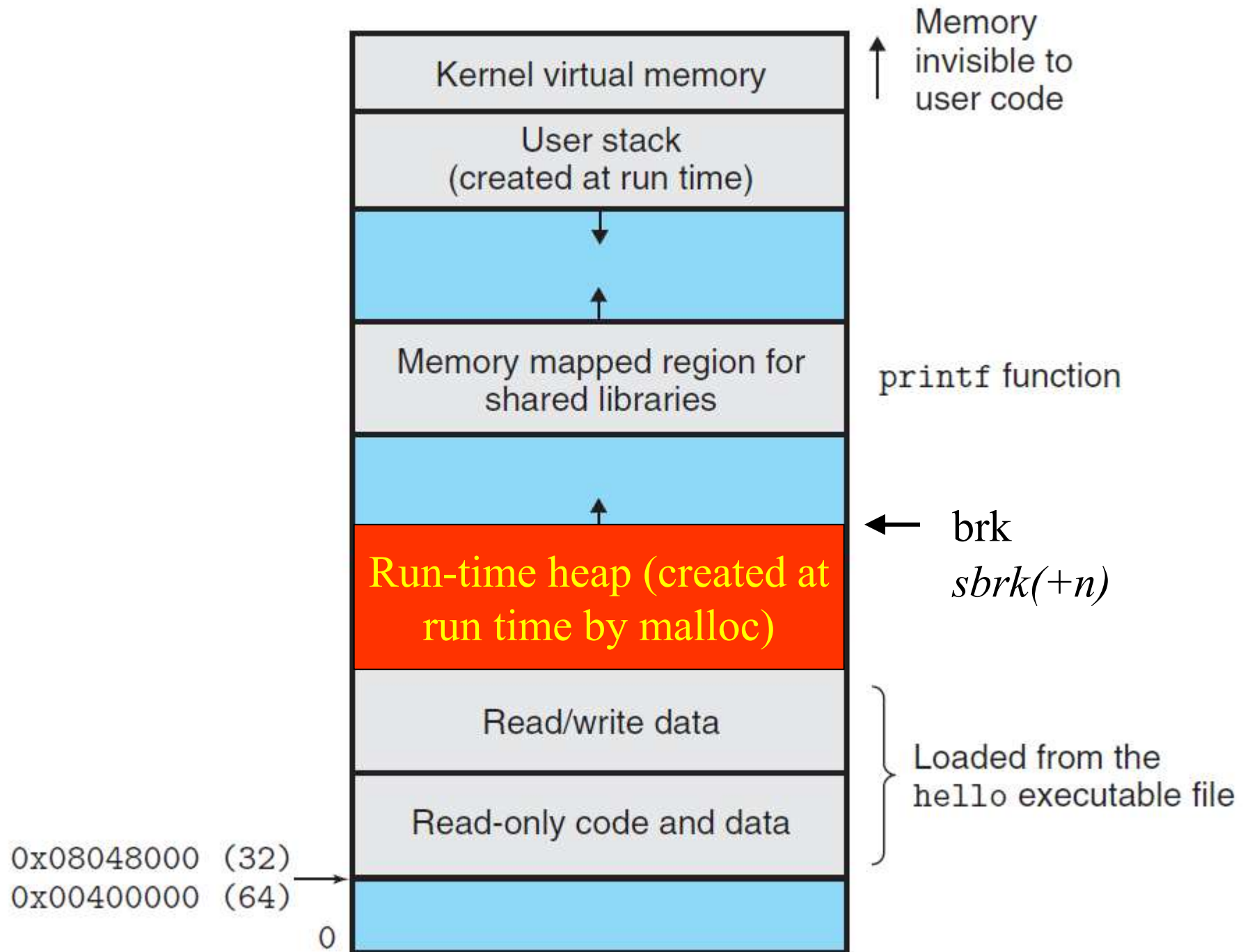
sbrk() : Where does Heap Memory Come From

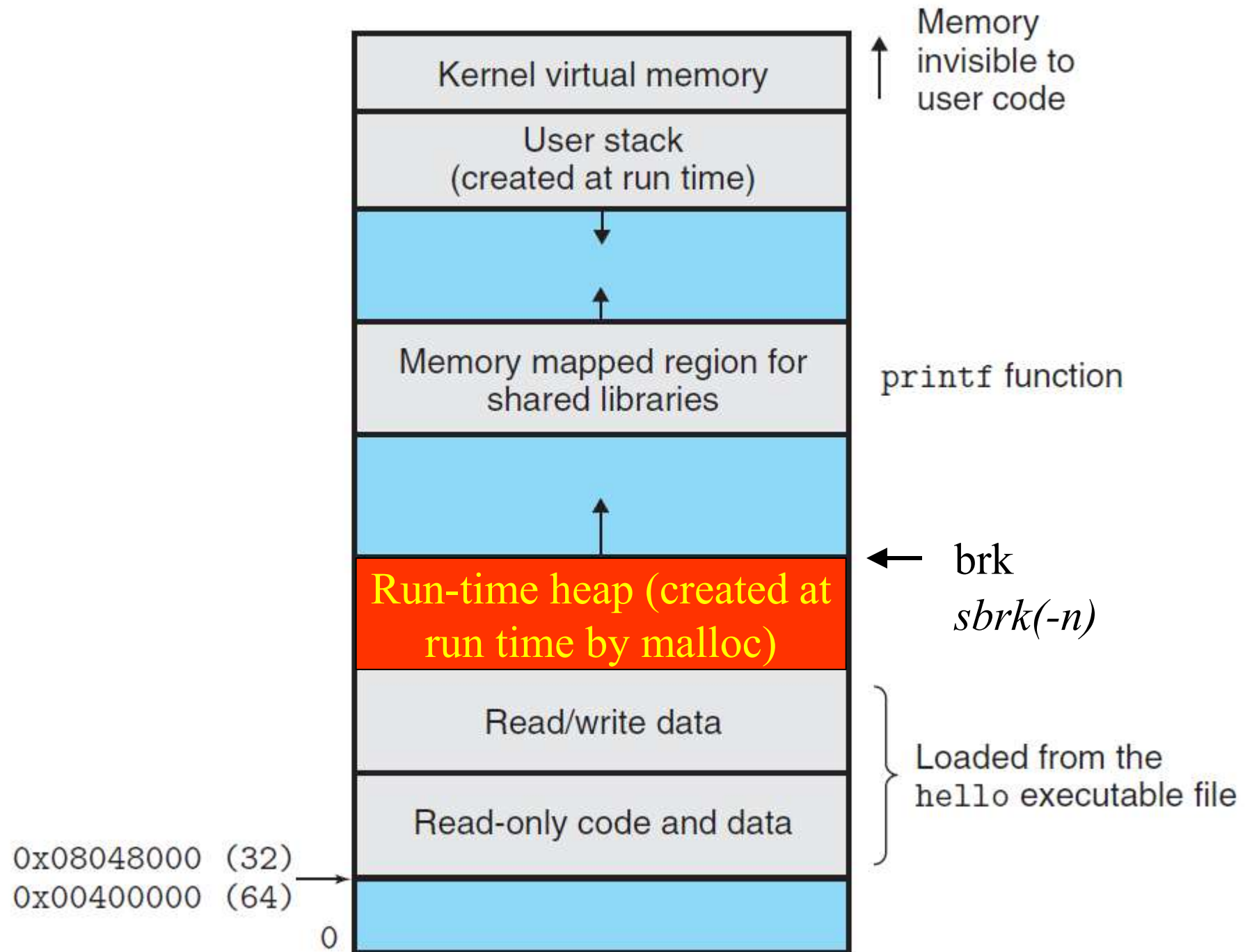
```
#include <unistd.h>
```

```
void *sbrk(int incr)
```

- If successful
 - It returns the old value of **brk**
- If unsuccessful
 - It returns -1
 - It sets `errno` to `ENOMEM`
- If `incr` is zero
 - It returns the current value
- `incr` can be a negative number

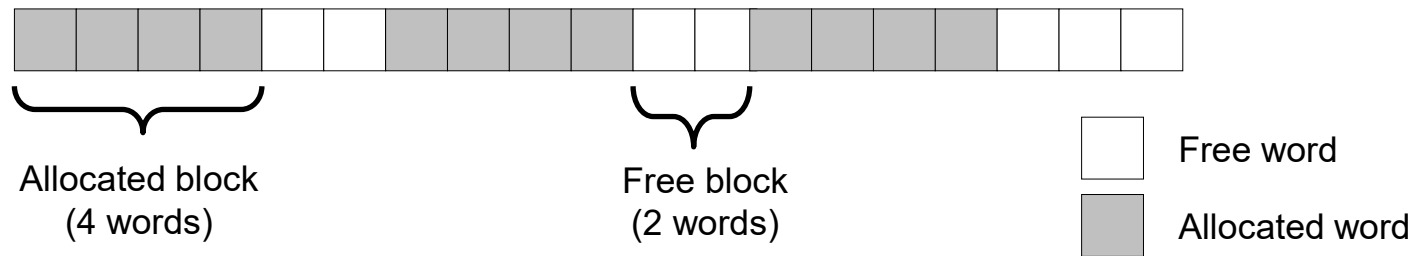






Assumptions

- The memory allocator uses `sbrk` to expand the heap and then manages the heap as a set of blocks
- Enforce double-word alignment on the return value of `malloc`
 - Each word holds 4 bytes
 - Q: What is the cause of this enforcement?



Allocation examples

```
p1 = malloc(4*sizeof(int))
```



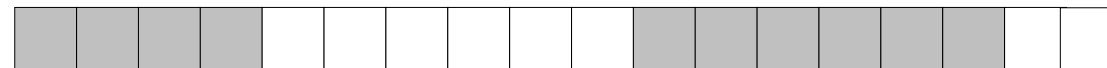
```
p2 = malloc(5*sizeof(int))
```



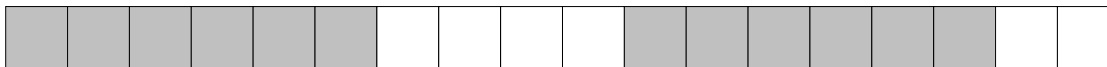
```
p3 = malloc(6*sizeof(long))
```



```
free(p2)
```



```
p4 = malloc(2*sizeof(int))
```



Allocator Requirements

- **Handle arbitrary sequence of requests**
 - No prior knowledge
- **Making immediate responses to requests**
 - No batching
- **Using only the heap**
- **Aligning blocks**
- **Not modifying allocated blocks**

Goals

- **Given some sequence of malloc and free requests:**
 - $R_0, R_1, \dots, R_k, \dots, R_{n-1}$
- **Want to maximize throughput and memory utilization**
 - These goals are often conflicting

Performance goals: throughput

- **Number of completed requests per unit time**
- **Example:**
 - 5,000 malloc calls and 5,000 free calls in 1 second
 - throughput is 10,000 operations/second

Performance goals: memory utilization

- Given some sequence of malloc and free requests:
 - $R_0, R_1, \dots, R_k, \dots, R_{n-1}$
- Def: aggregate payload P_k
 - malloc(m) results in a block with a **payload** of m bytes
 - After request R_k has completed, the **aggregate payload** P_k is the sum of currently allocated payloads

Performance goals: memory utilization

- **Def: current heap size is denoted by H_k**
 - Assume that H_k is monotonically nondecreasing (can be relaxed)
- **Def: memory utilization:**
 - After k requests, **memory utilization** is:
$$U_k = P_k / H_k$$

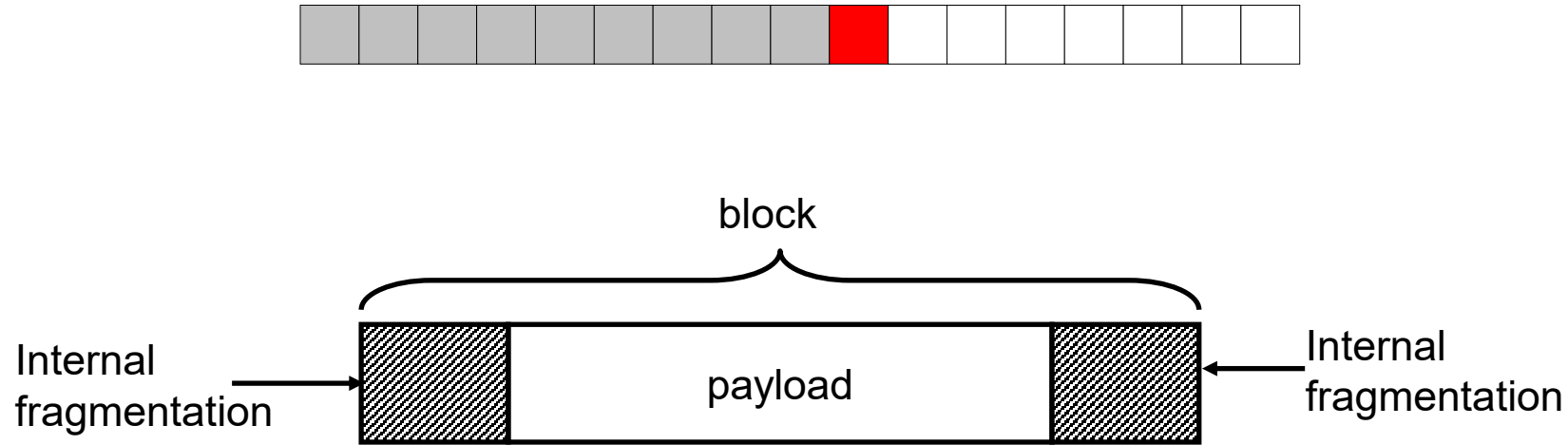
$$U_k = (\max_{i \leq k} P_i) / H_k \text{ (peak memory utilization)}$$

Fragmentation

- **Poor memory utilization caused by fragmentation**
- **Two forms of fragmentation**
 - Internal fragmentation
 - External fragmentation

Internal Fragmentation

- **Internal fragmentation**
 - For some block, internal fragmentation is the difference between the block size and the payload size



Internal Fragmentation

- **Internal fragmentation**
 - Is caused by overhead of maintaining heap data structures, padding for **alignment** purposes, or explicit **policy decisions** (e.g., not to split the block)
 - Depends only on the pattern of previous requests, and thus is easy to measure

External Fragmentation

- Occurs when there is enough aggregate heap memory, but no single free block is large enough

The case of external fragmentation:



```
p4 = malloc(6*sizeof(int))
```

The case of no external fragmentation:

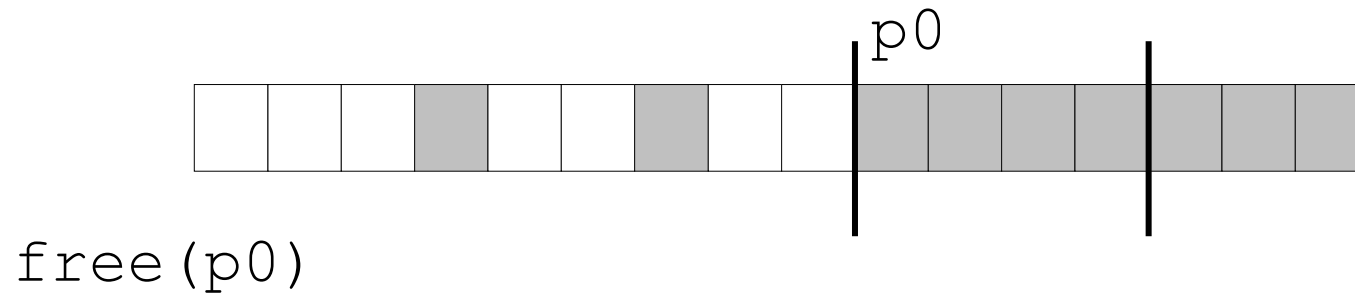


```
p4 = malloc(2*sizeof(int))
```

External fragmentation depends on the pattern of future requests, and thus is difficult to measure

Implementation Issues

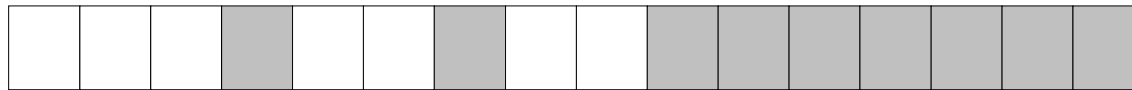
1. How do we know how much memory to free just given a pointer?
2. How do we keep track of the free blocks?



Implementation Issues

3. How do we pick a block to use for allocation?

- many might fit?



```
p1 = malloc(sizeof(int))
```

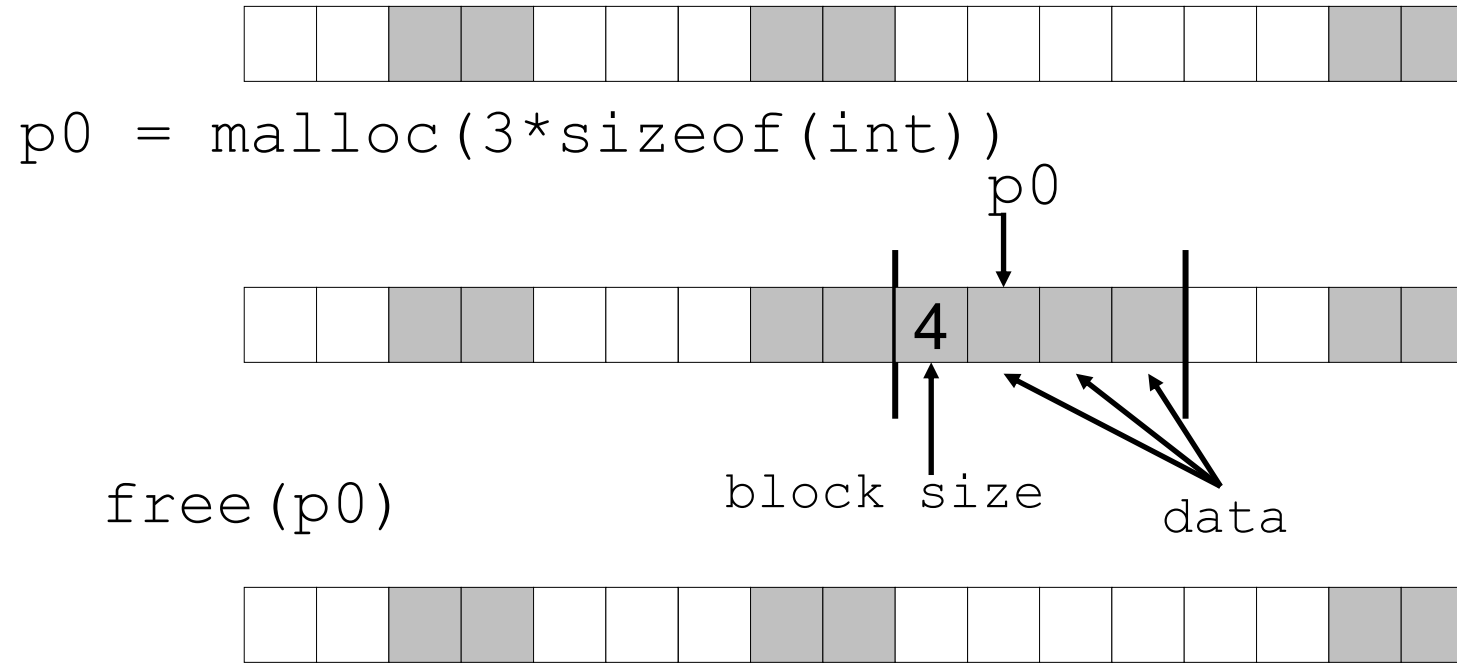
4. How to deal with the extra space when allocating a structure that is smaller than the free block it is placed in?

5. How do we reinsert freed block?

Knowing how Much to Free

- **Standard method**
 - keep the length of a structure in the word preceding the structure
 - This word is often called the **header field** or **header**
 - requires an extra word for every allocated structure

Knowing how Much to Free

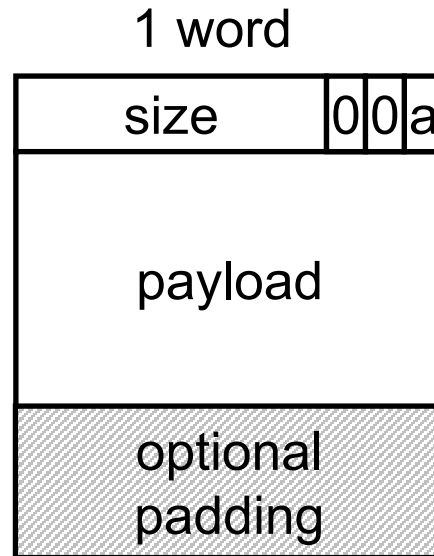


Implicit List

- **Need to identify whether each block is free or allocated**
 - Can use extra bit
 - Bit can be put in the same word as the size if block sizes are always multiples of 8 (mask out low order bit when reading size)

Implicit list

Format of
allocated and
free blocks

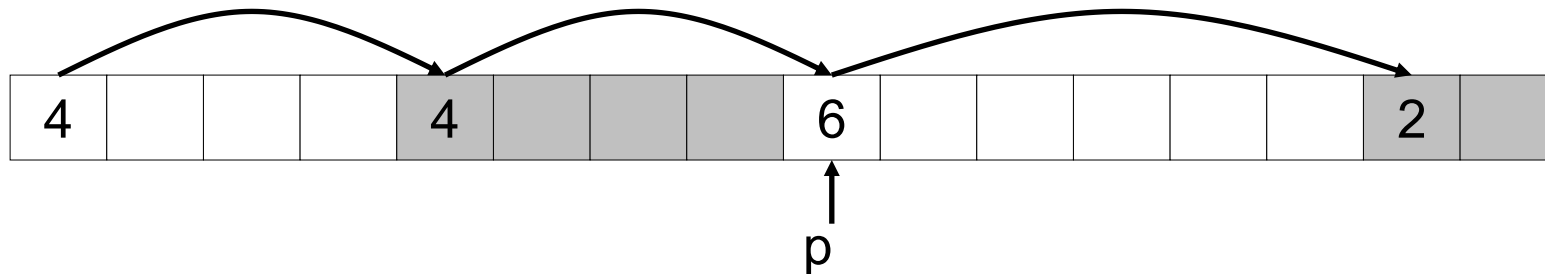


a = 1: allocated block

a = 0: free block

size: block size

payload: application data
(allocated blocks only)



Finding a Free Block

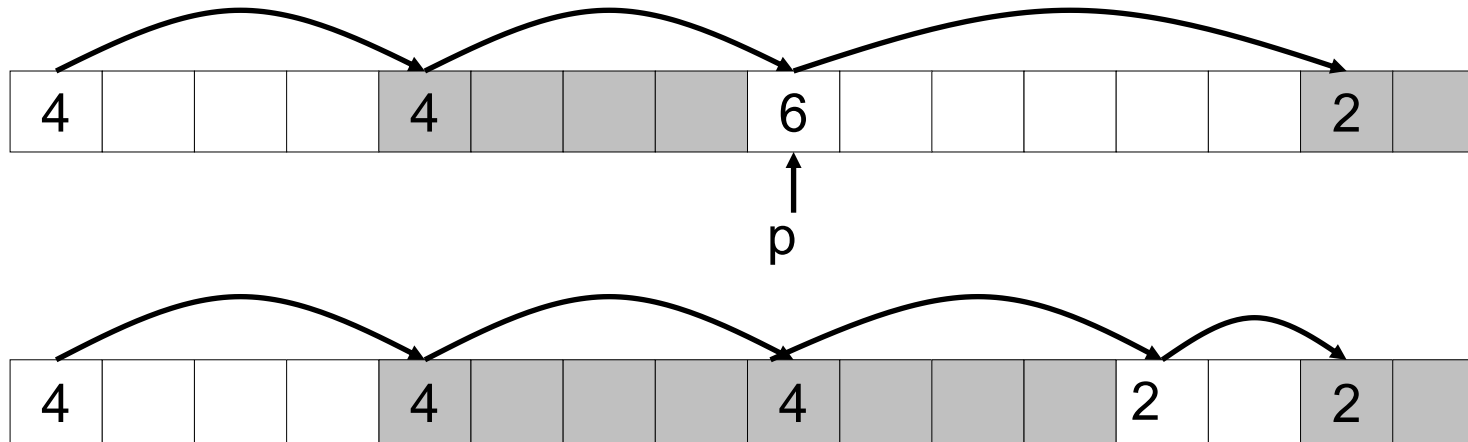
- **First fit:**
 - Search list from beginning, choose first free block that fits
 - Can take linear time in total number of blocks (allocated and free)
 - In practice it can cause “splinters” at beginning of list

Finding a Free Block

- **Next fit:**
 - Like first-fit, but search list from location of end of previous search
 - Research suggests that fragmentation is worse
- **Best fit:**
 - Search the list, choose the free block with the closest size that fits
 - Keeps fragments small --- usually helps reducing fragmentation
 - Will typically run slower than first-fit

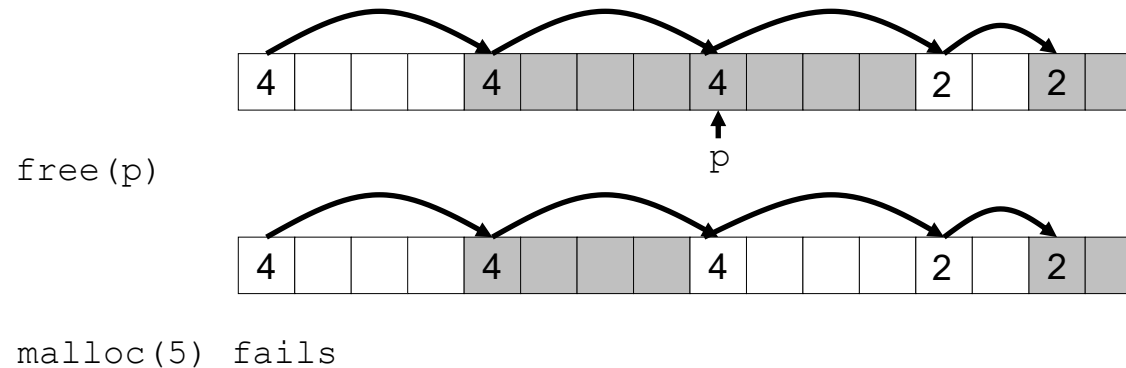
Allocating in a free block

- **Allocating in a free block - splitting**
 - Since allocated space might be smaller than free space, we might want to split the block



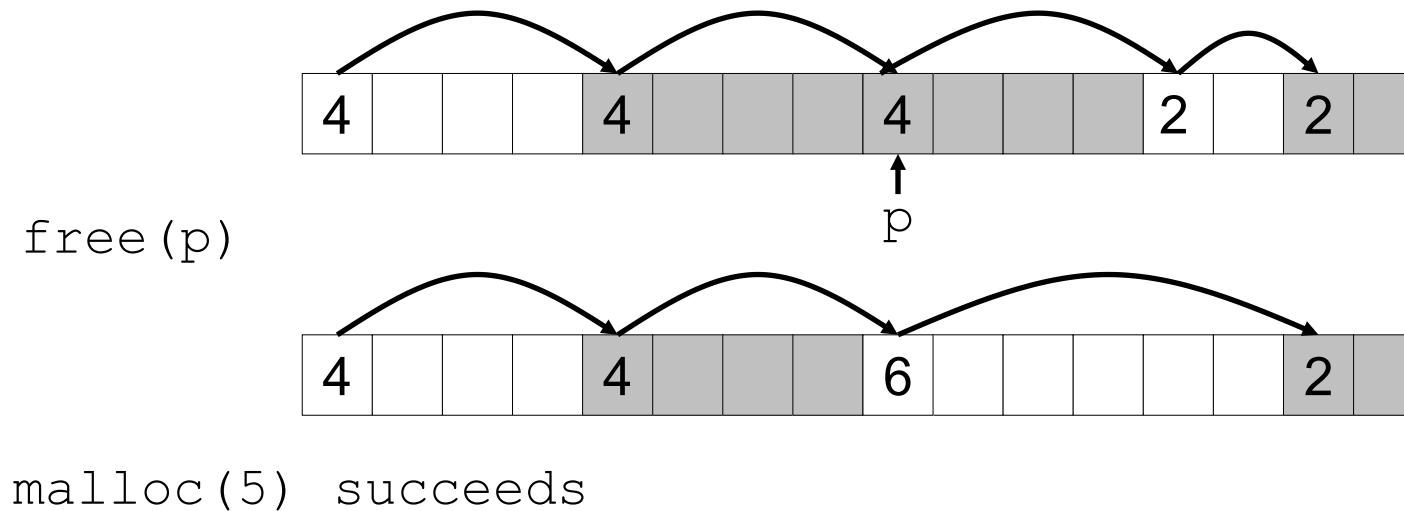
Freeing a block

- **Simplest implementation:**
 - Only need to clear allocated flag
 - But can lead to “false fragmentation”
 - There is enough free space, but the allocator won’t be able to find it



Coalescing

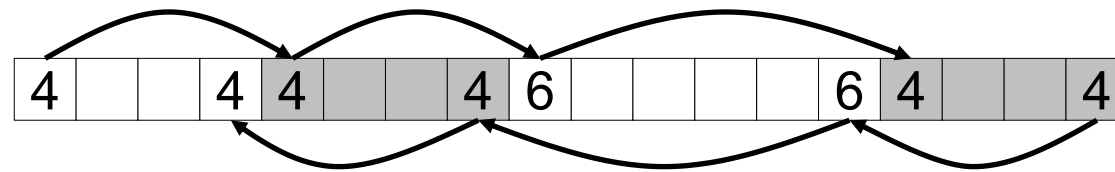
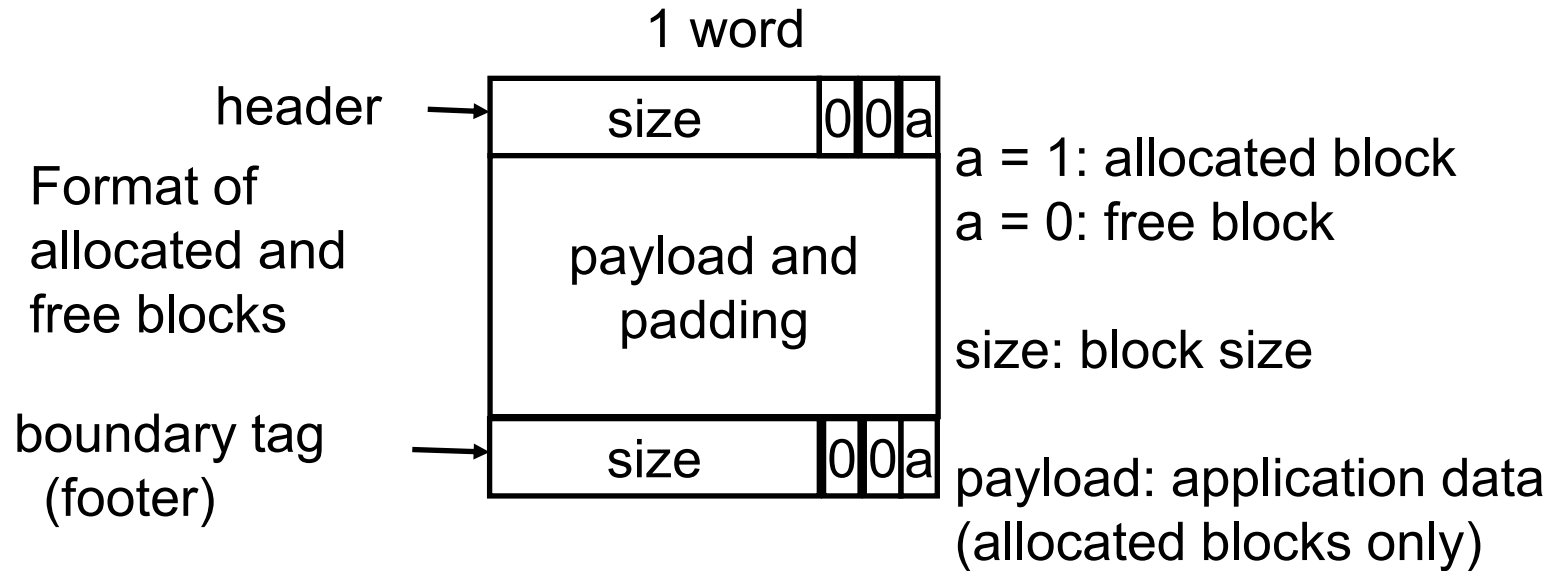
- Join with next and/or previous block if they are free
 - Coalescing with next block
 - Q: But how do we coalesce with previous block?



Bidirectional Coalescing

- **Boundary tags**
 - Replicate size/allocated word at bottom of free blocks
 - Allows us to traverse the “list” backwards, but requires extra space
 - An important and general technique!

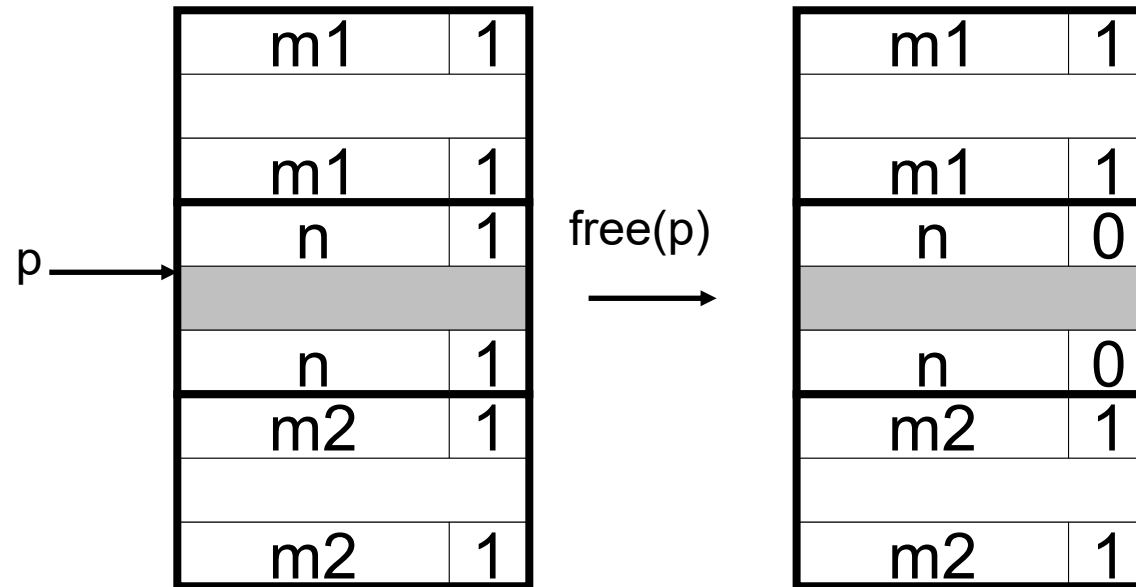
Bidirectional Coalescing



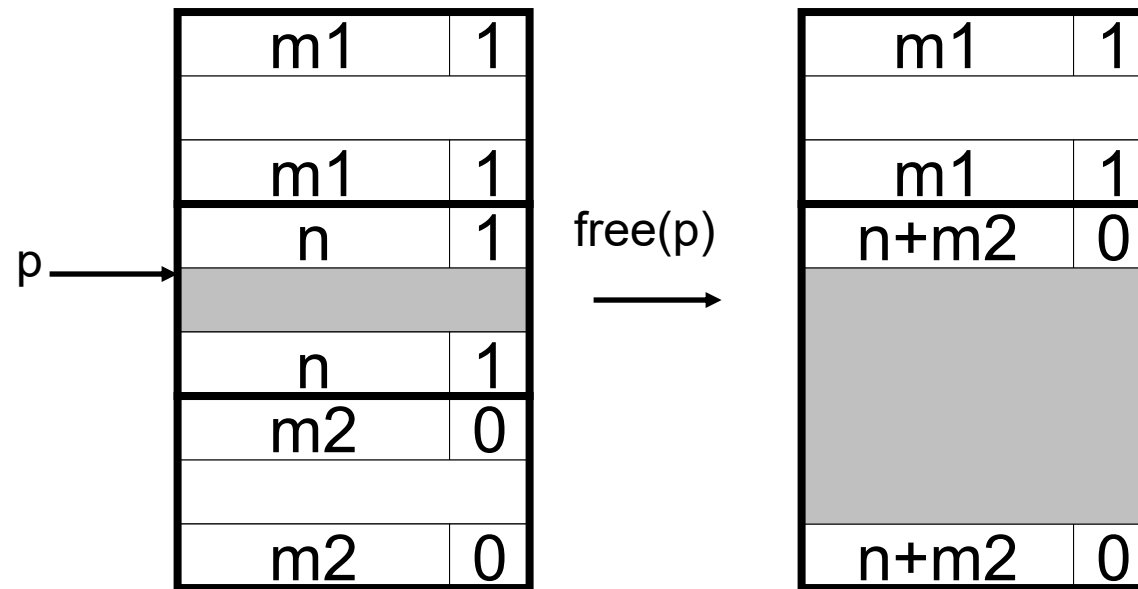
Constant Time Coalescing



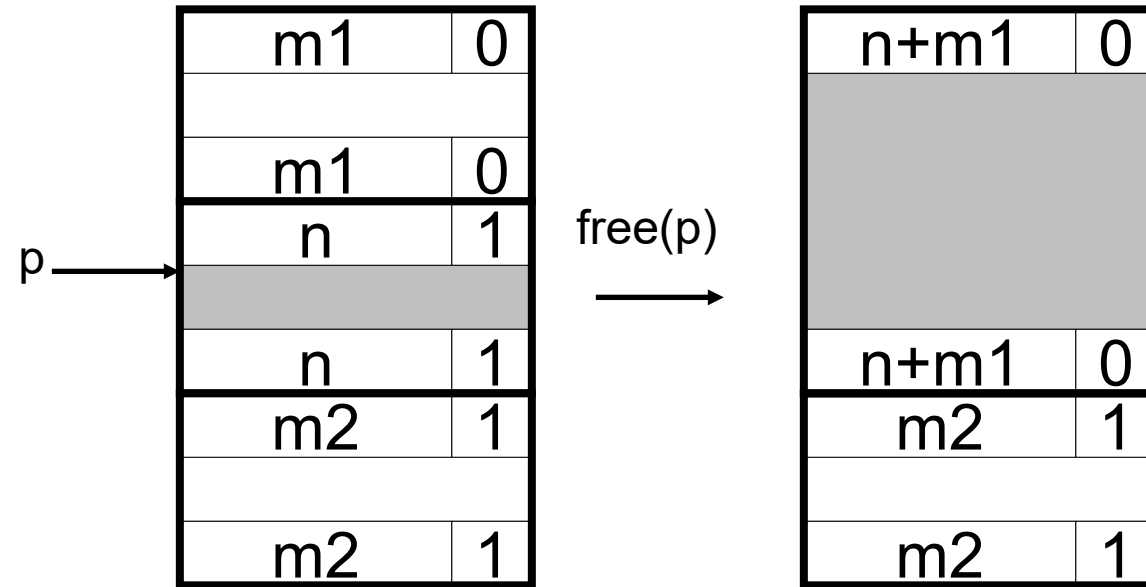
Constant Time Coalescing (Case 1)



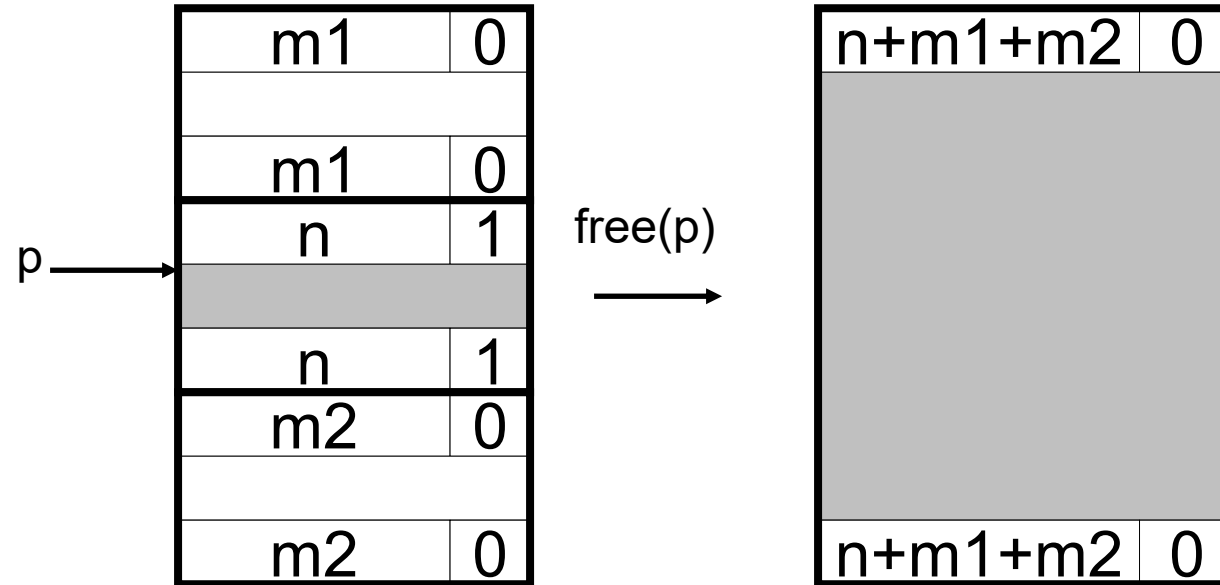
Constant Time Coalescing (Case 2)



Constant Time Coalescing (Case 3)



Constant Time Coalescing (Case 4)



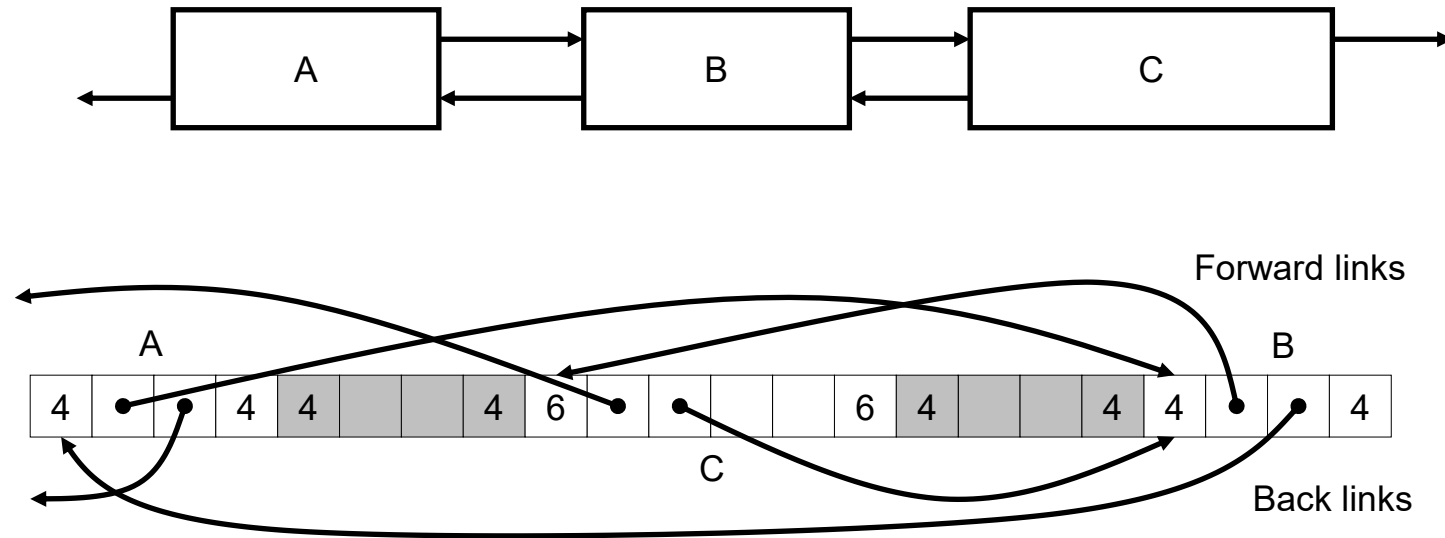


HOW TO ACHIEVE BETTER PERFORMANCE?

Explicit Free Lists

- **Explicit list among the free blocks using pointers within the free blocks**
- **Use data space for link pointers**
 - Typically doubly linked
 - Still need boundary tags for coalescing
 - It is important to realize that links are not necessarily in the same order as the blocks

Explicit Free Lists



Freeing with Explicit Free Lists

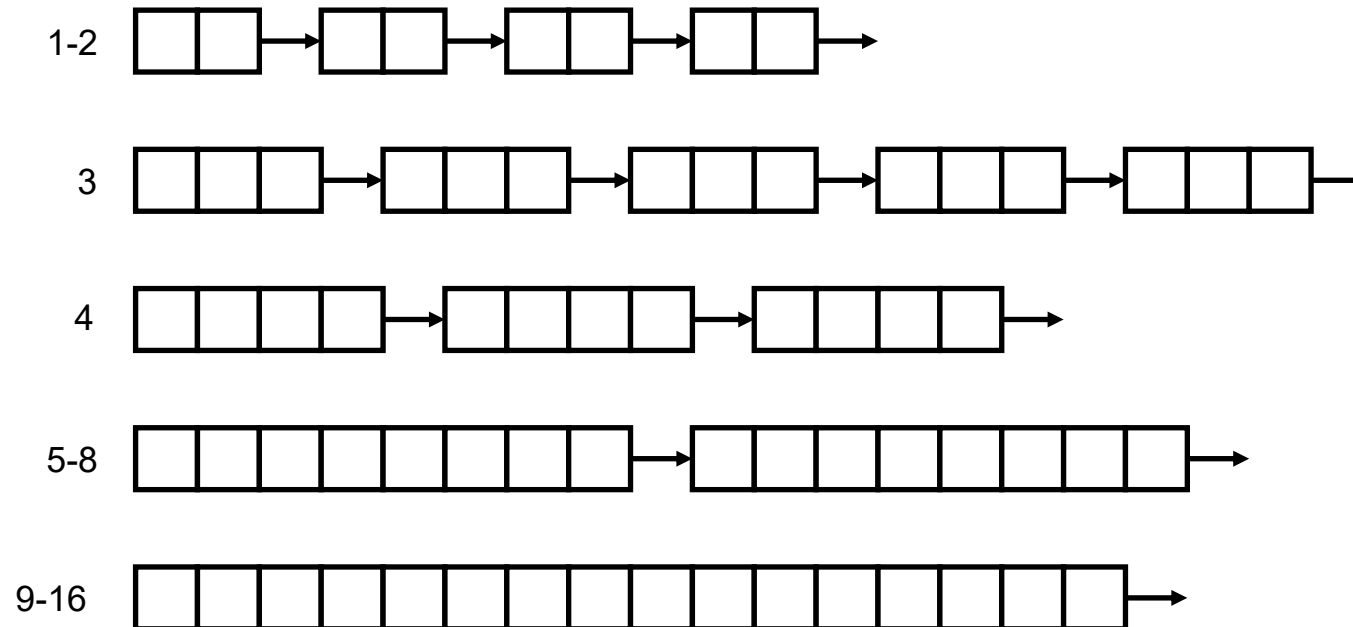
- **Where to put the newly freed block in the free list**
 - LIFO (last-in-first-out) policy
 - insert freed block at the beginning of the free list
 - pro: simple and constant time
 - con: studies suggest fragmentation is worse than address ordered

Freeing with Explicit Free Lists

- **Where to put the newly freed block in the free list**
 - Address-ordered policy
 - insert freed blocks so that free list blocks are always in address order
 - i.e., $\text{addr}(\text{pred}) < \text{addr}(\text{curr}) < \text{addr}(\text{succ})$
 - con: requires search
 - pro: studies suggest fragmentation is better than LIFO

Segregated Storage

- **Each size “class” has its own collection of blocks**
 - Often have separate collection for every small size (2,3,4,...)
 - For larger sizes typically have a collection for each power of 2



Simple Segregated Storage

- **Separate free lists for each size class & No splitting**
- **To allocate a block of size n:**
 - if free list for size n is not empty,
 - allocate first block on list (note, list can be implicit or explicit)
 - if free list is empty,
 - get new heap memory
 - create new free list from all blocks in page
 - allocate first block on list
 - constant time
- **To free a block:**
 - add to free list
- **Tradeoffs: fast, but can fragment badly**

Segregated Fits

- **Array of free lists, each one for some size class**
- **To allocate a block of size n :**
 - search appropriate free list for block of size $m > n$
 - if an appropriate block is found:
 - split block and place fragment on appropriate list (optional)
 - if no block is found, try next larger class
 - repeat until block is found
 - if no blocks is found in all classes, try more heap memory

Segregated Fits

- **To free a block:**
 - coalesce and place on appropriate list (optional)
- **Tradeoffs**
 - faster search than sequential fits (i.e., log time for power of two size classes, $O(\text{NUM_LIST})$)
 - controls fragmentation
 - A simple first-fit approximates a best-fit over entire heap
 - coalescing can increase search times
 - deferred coalescing can help

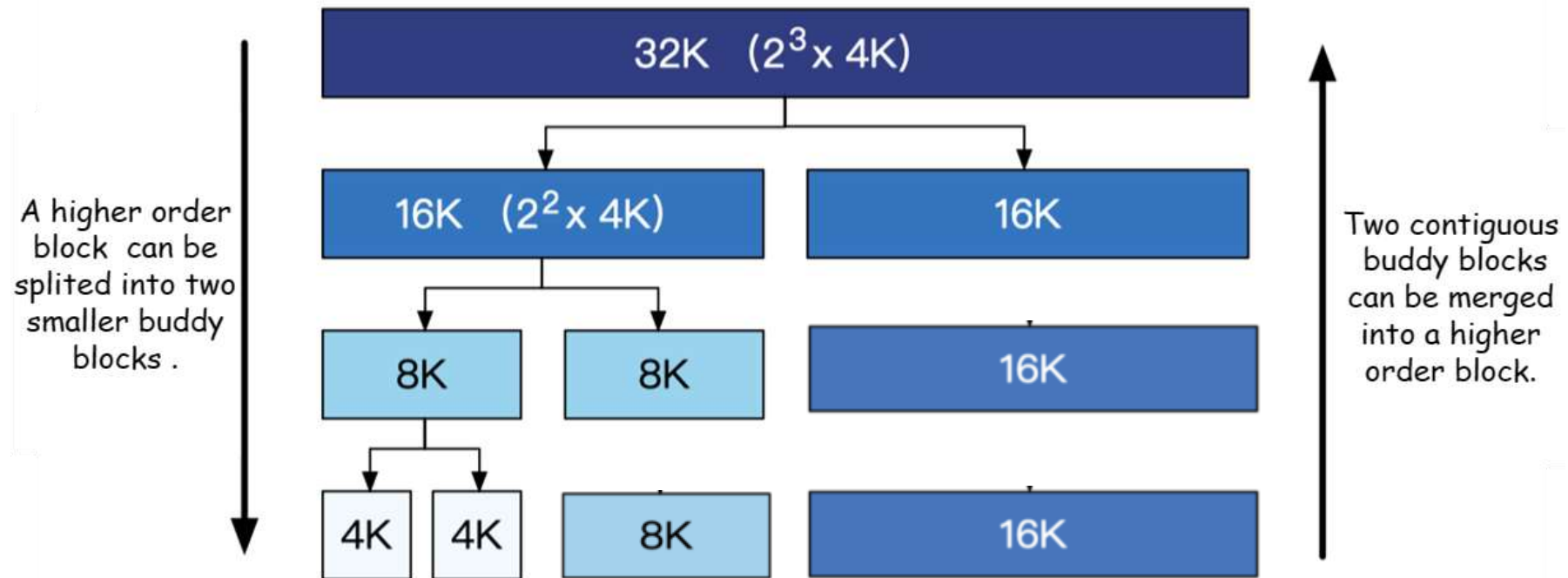
Buddy Systems

- **A special case of segregated fits**
 - Each size is power of 2
- **Initialize**
 - A heap of size 2^m

Buddy Systems

- **Allocate**
 - Roundup to power of 2 such as 2^k
 - Find a free block of size 2^j ($k \leq j \leq m$)
 - Split the block in half until $j=k$
 - Each remaining half block (buddy) is placed on the appropriate free list
- **Free**
 - Continue coalescing with the free buddies

Buddy Systems

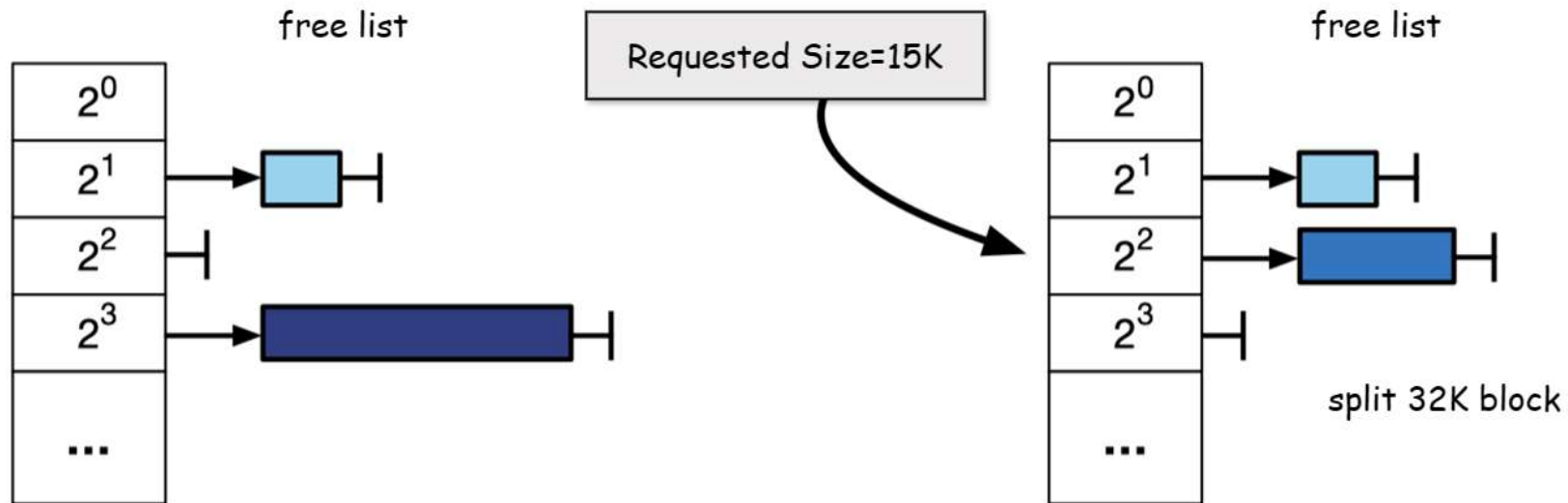


Recursively split and merge

OS kernel uses this way to allocate/free physical pages

Allocate

- To allocate a block with m physical pages, buddy system will find an appropriate block that contains 2^n pages and satisfies $2^{n-1} < m \leq 2^n$



Free

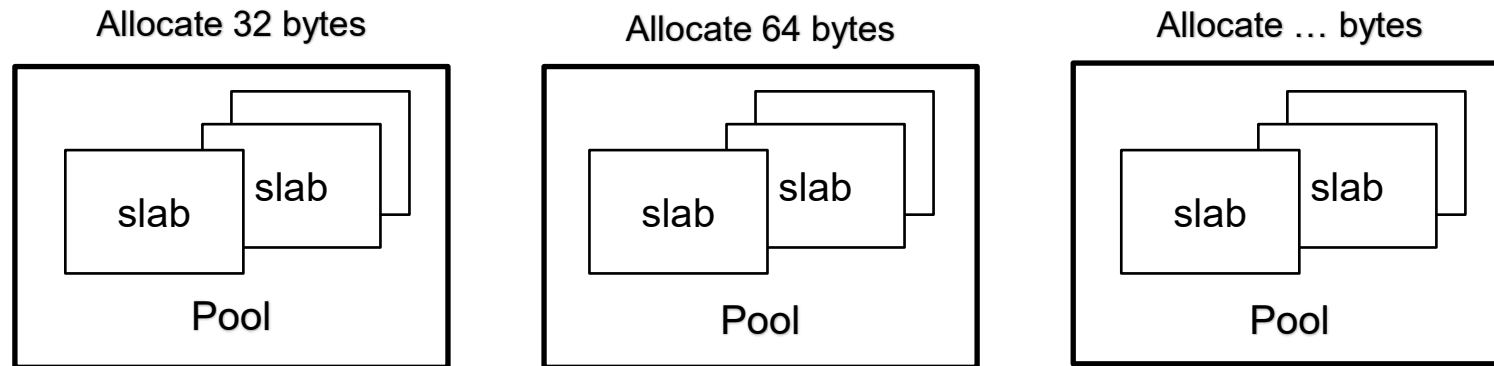
- **Efficiently find the buddy of block**
 - Addresses of buddy blocks only differ in exactly **one bit** position
 - Position of differed bit is up to the **size of block**
- **Example:**
 - Block A (0-8K) Block B (8-16K)
 - Addr of block A is 0x0 , Addr of block B is 0x2000
 - Only differ in **13**-th bit with block size 8K (2^{13})

SLAB Allocator

- **Observation**
 - Sizes of frequently allocated objects can be known
- **Main idea**
 - Get contiguous pages first from the OS (slab)
 - Carved into small slots of fixed size
 - Usually 2^n bytes each slot (Generally $3 \leq n < 12$)
 - Other sizes can be used for specific data structure to reduce internal fragment

Design of Resource Pool

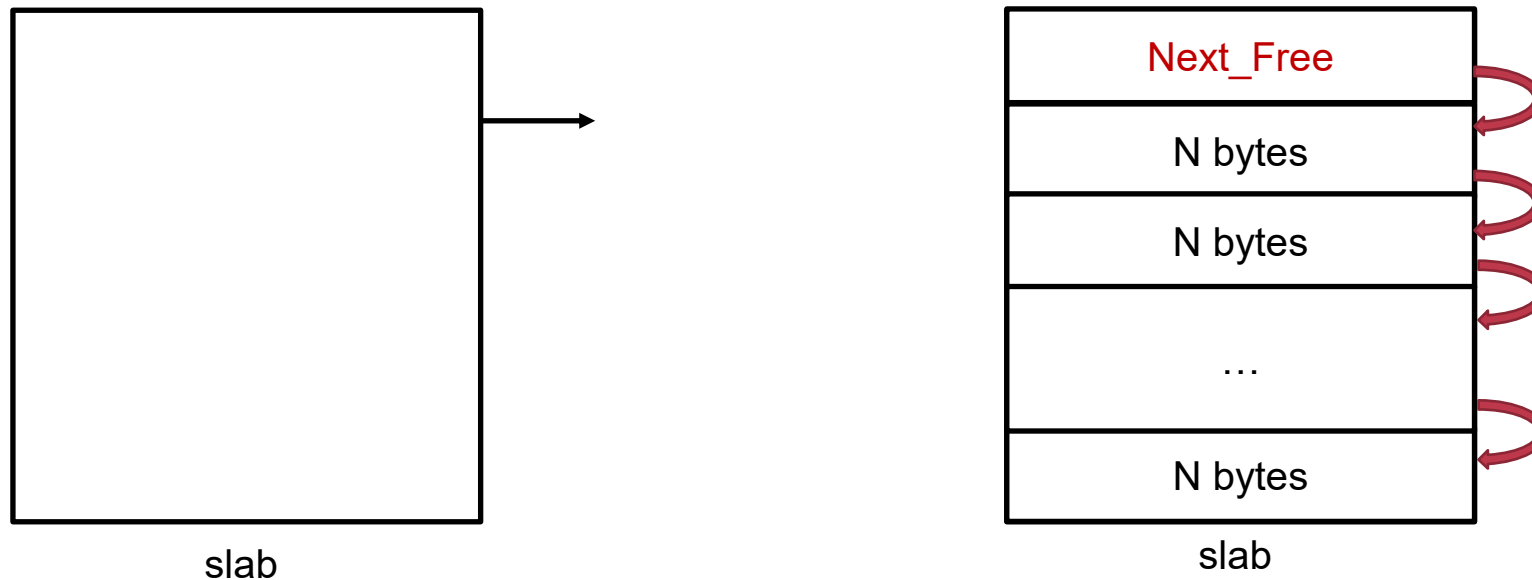
- **Only allocate blocks with fixed size.**
- **SLAB allocator use separate memory pools for each size of blocks**
 - Use **best fit** to choose memory pool



Initialize free list

How to distinguish free slots? Free list!

Use a free slot when allocating.

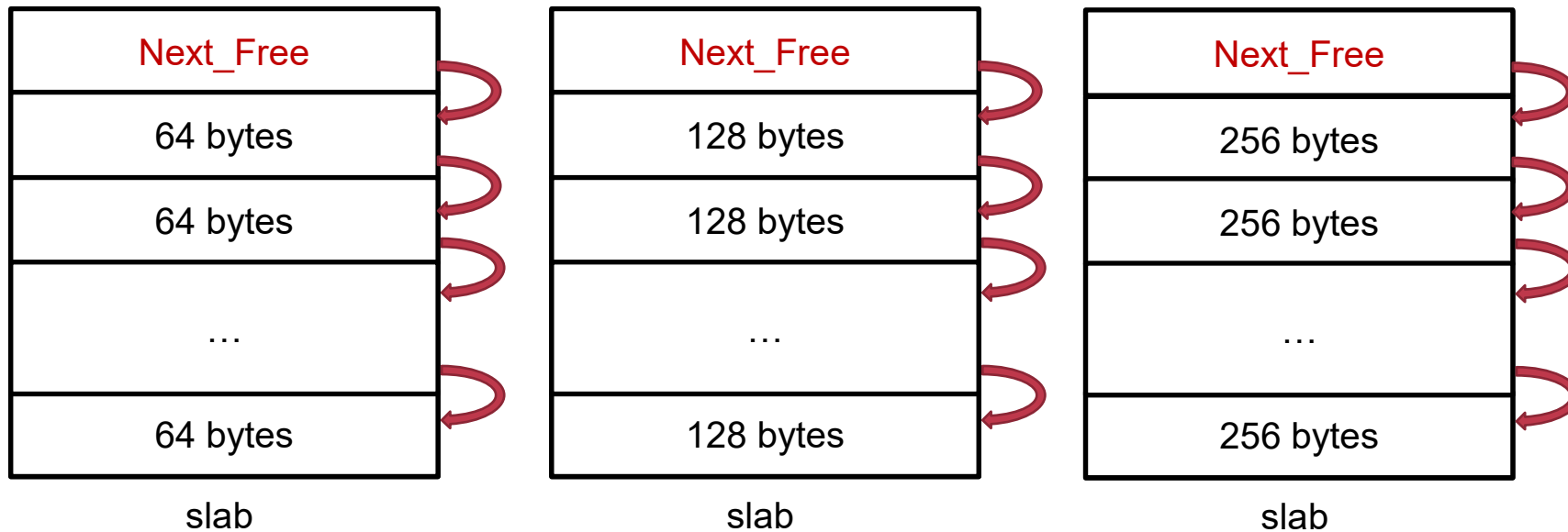


Allocate and free

Allocate N bytes:

1. locate best-fit memory pool (choose one slab in the pool)
2. take the first slot pointed by Next_Free from slab

Free: let Next_Free point to the slot to free



Q: How to find Next_Free?

How to Organize Different Slabs in One Pool

Maintain two ptr for each size of slot:

- current points to only one slab
 - used for allocating
 - on-demand updating
- partial points to available slab
 - free the slab if it is empty

Q1: How to choose slot sizes?

Q2: Time complexity of allocating and free?

Q3: Where is the full slab?

Q4: Why we need to free the empty slab?

